

CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA DE MINAS GERAIS
CAMPUS CONTAGEM - INFORMÁTICA III

Caio Vitor, Carlos Antonio, Lucas Galarani

RELATÓRIO ROBOCODE

CONTAGEM - MG
2026

SUMÁRIO

1. INTRODUÇÃO..... 3
1.1 Objetivo Geral..... 4
1.2 Objetivos Específicos..... 4
1.3 Requisitos Funcionais..... 4
1.4 Requisitos Não Funcionais..... 5
2. DESENVOLVIMENTO..... 5
2.1 Etapa 1..... 5
2.1.1 Padrão State..... 5
2.1.2 Desenvolvimento do reconhecimento..... 6
2.1.3 Torneio e Avaliação Experimental..... 7
2.1.4 Diagrama de Classes..... 8
2.2 Etapa 2 - Torneio..... 9
2.2.1 Detalhamento das Estratégias Avançadas..... 9
2.2.2 Aplicação Prática e Vantagens Táticas..... 10
2.2.3 Limitações e Desvantagens da Estratégia Senoidal..... 10
2.2.4 Análise de Vulnerabilidades: Testes contra Estratégias de Cerco..... 11
2.2.5 Adaptação Estratégica: Manobra de Evasão Tática..... 12
2.2.6 Evolução Ofensiva: Implementação do Tiro de Previsão..... 13
2.2.7 Otimização Final: Aprimoramento da Mira Preditiva e Superação de Limitações. 15
3. CONCLUSÃO..... 17
4. REFERÊNCIAS..... 19

1. INTRODUÇÃO

Este trabalho tem como objetivo demonstrar/comentar o desenvolvimento de um robô digital para competições. O robô em questão é utilizado para o programa Robocode, programa que permite a compilação de novas máquinas, geração de partidas ou campeonatos e etc.

O Robocode é uma plataforma educacional que permite a criação de robôs virtuais representados por tanques de batalha. Cada robô deve ser programado para perceber o ambiente, tomar decisões e executar estratégias de movimentação, busca e combate.

Por meio deste projeto pretende-se praticar conceitos de POO e Padrões de Projeto aprendidos durante as aulas de LP2 e LLP2.

No mundo, o Robocode é muito presente em campeonatos com diversas categorias de automatôs. Sua distincção se assemelha aos modelos de luta real onde há pesos diferentes e uma faixa de pesos competem entre si, modelo muito praticado no esporte de MMA no UFC. No roboode esses robôs são “pesados” pelo peso total na hora de sua compilação, seguindo a seguinte categoria:

NanoBot	até 249 Bytes
MicroBot	até 749 bytes
MiniBot	até 1499 bytes
MegaBot	Não há restrição

Portanto, neste trabalho iremos tratar sobre o nosso robô que deve obrigatoriamente seguir a categoria de MiniBot, assim como todos os grupos que estarão disputando o campeonato e discutir a estratégia escolhida, dados de eficiência, decisões, outros assuntos.

1.1 Objetivo Geral

Desenvolver um robô de batalha utilizando a plataforma Robocode, aplicando conceitos de Programação Orientada a Objetos e modelagem de comportamento.

1.2 Objetivos Específicos

- Implementar um robô funcional utilizando Robocode;
- Aplicar conceitos de POO;
- Utilizar herança e polimorfismo;
- Modelar comportamentos utilizando máquina de estados finitos;
- Desenvolver estratégias de busca, movimentação e combate;
- Estruturar o código de forma modular;
- Utilizar Git/GitHub para versionamento;
- Produzir relatório técnico em conformidade com as normas da ABNT.

1.3 Requisitos Funcionais

Todos os requisitos essenciais ao robô são cumpridos e não houve nada de complicações para o funcionamento.

RF01 – Inicialização
O robô deve ser carregado e executado corretamente pelo Robocode.
RF02 – Uso de AdvancedRobot
O robô deve obrigatoriamente estender a classe AdvancedRobot.
RF03 – Reconhecimento do robô lp2.alisson
O sistema deve ser capaz de identificar a presença do robô lp2.alisson durante a etapa de reconhecimento.
RF04 – Navegação até a parede
O robô deve localizar e alcançar a parede mais próxima.
RF05 – Posicionamento seguro
O robô deve posicionar-se fora da região protegida do Border Guard.
RF06 – Patrulhamento da arena
O robô deve percorrer completamente o campo de batalha.
RF07 – Localização do alvo
O robô deve localizar o robô lp2.alisson após concluir o patrulhamento.
RF08 – Destruição do alvo
O robô deve eliminar o robô lp2.alisson.
RF09 – Participação em torneio
O robô deve ser capaz de competir contra outros robôs sem intervenção humana.
RF10 – Máquina de Estados
O comportamento do robô deve ser modelado utilizando uma máquina de estados finitos.
RF11 – Estratégias de combate
O robô deve implementar pelo menos uma estratégia de movimentação e uma estratégia de ataque.

RF12 – Utilização de GitHub
O projeto deve ser versionado e compartilhado com o professor

1.4 Requisitos Não Funcionais

Todos estão em perfeito estado e inclusive há um complemento de que o robô funciona no Linux

RNF01 – Organização do Código
O código deve ser modular, legível e devidamente comentado.
RNF02 – Orientação a Objetos
A solução deve aplicar adequadamente conceitos de encapsulamento, herança e polimorfismo.
RNF03 – Desempenho
O robô deve operar sem comprometer significativamente o desempenho da simulação.
RNF04 – Rastreabilidade
As estratégias implementadas devem estar descritas no relatório.
RNF05 – Reprodutibilidade
Os testes descritos no relatório devem permitir a reprodução dos resultados obtidos.
RNF06 – Compatibilidade
O projeto deve executar corretamente utilizando Java 25 e a versão do Robocode utilizada em laboratório

2. DESENVOLVIMENTO

Nesta seção haverá a discussão sobre as etapas requisitadas pelo documento orientador deste trabalho.

2.1 Etapa 1

2.1.1 Padrão State

Na primeira etapa do desenvolvimento de máquinas é feito o papel de reconhecimento do ambiente e dos inimigos, além da identificação de um inimigo teste chamado “alisson”. Aqui se tem a criação principal das classes de funcionamento do robô, funções, estrutura e programa principal.

Antes de tratar sobre a criação de um robô, é necessário falar sobre Padrões de Projeto e a escolha pelo padrão utilizado. Padrões de Projeto ou *Design Patterns* são soluções comuns e reutilizaveis para problemas comuns no desenvolvimento de programas. Dentre os diversos padrões de projeto utilizados na programação, o padrão escolhido e considerado adequado para esse tipo de aplicação e também para jogos é o *State* ou Estado.

O *State* é um padrão que permite que um objeto altere o seu comportamento quando seu estado interno se altera. Algo semelhante a alterar valores de uma variavel (só a nivel de exemplificação). A ideia principal é que, em qualquer dado momento, há um número *finito* de *estados* que um programa possa estar. Dentro de qualquer estado único, o programa se comporta de forma diferente, e o programa pode ser trocado de um estado para outro instantaneamente. Contudo, dependendo do estado atual, o programa pode ou não trocar para outros estados. Essas regras de troca, chamadas *transições*, também são finitas e pré determinadas por quem programa.

O padrão State sugere que você crie novas classes para todos os estados possíveis de um objeto e extraia todos os comportamentos específicos de estados para dentro dessas classes. Ao invés de implementar todos os comportamentos por conta própria, o objeto original, chamado *contexto*, armazena uma referência para um dos objetos de estado que representa seu estado atual, e delega todo o trabalho relacionado aos estados para aquele objeto.

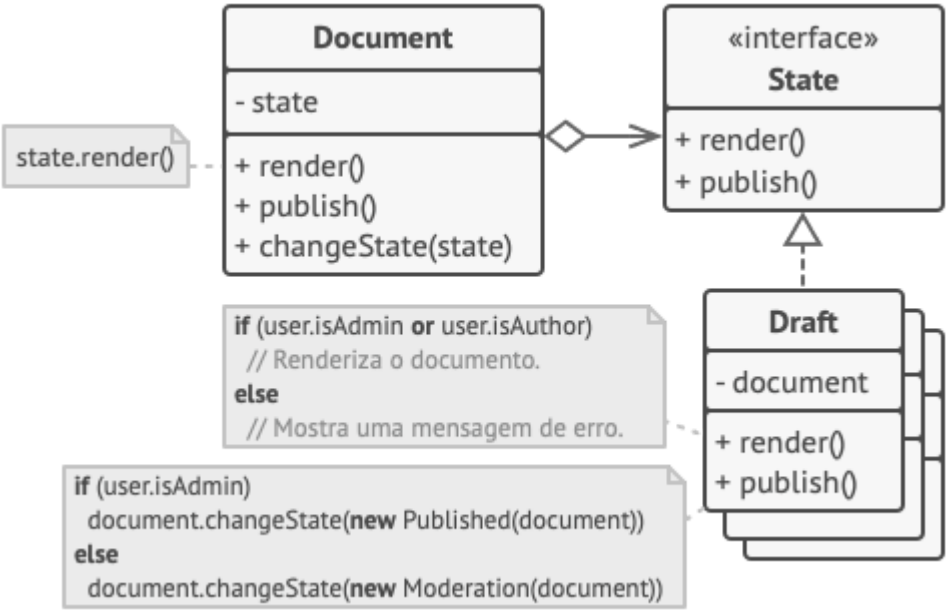


FIGURA 01 - funcionamento de um state, fonte: State

Essa estrutura pode ser parecida com o padrão Strategy, mas há uma diferença chave. No padrão *State*, os estados em particular podem estar cientes de cada um e iniciar transições de um estado para outro, enquanto que estratégias quase nunca sabem sobre as outras estratégias. Sendo por isso o mais adequado para robôs, já que se manipula diretamente seus estados cumprindo uma requisição específica.

2.1.2 Desenvolvimento do reconhecimento

Explicado o padrão utilizado neste projeto, o robô em primeira etapa e condicional ao entrar na arena, deveria fazer um reconhecimento do campo e procurar por uma máquina inimiga chamada “alisson”. Logo após identificado, ele deve seguir algumas etapas. A tabela abaixo mostra também quais foram cumpridos.

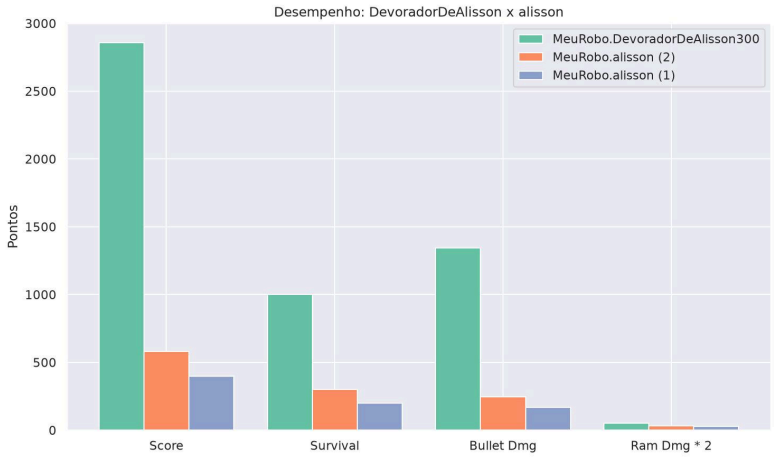
Localizar a parede mais próxima;	Cumprido✅
Deslocar-se até essa parede;	Cumprido✅
Posicionar-se fora da região protegida do Border Guard (Sentry Border);	Cumprido✅
Realizar uma volta completa pelo campo de batalha;	Cumprido✅
Dirigir-se ao centro da arena;	Cumprido✅
Localizar o robô lp2.alisson;	Cumprido✅
Posicionar o canhão em direção ao alvo;	Cumprido✅
Destruir o robô identificado	Cumprido✅

Seguindo os passos em ordem respectivas como citado acima, é obrigatório que se encontrado, ele deve eliminar o alvo. Estes passos não são necessariamente executados na etapa 2 (Competição), pois pode não haver o alvo, então ele terá um sistema condicional.

Essa etapa serve para o teste básico do robô e tem suas estatísticas padronizadas conforme abaixo. Por esses passos permite-se conhecer um pouco do desenvolvimento de robocode, ângulos e comandos. Passos que podem ser utilizados na etapa 2, caso seja julgado eficiente.

2.1.3 Torneio e Avaliação Experimental

Para validar o funcionamento inicial e o cumprimento dos requisitos funcionais da **Etapa 1** (que exigiam a localização, aproximação e destruição do robô lp2.alisson), realizou-se uma bateria de testes controlados na arena do Robocode. O gráfico de desempenho gerado a partir dos logs de combate (*Figura COLOCAR O NUMERO*) ilustra de forma clara a superioridade tática do robô desenvolvido pela equipe (MeuRobo.DevoradorDeAlisson300) em comparação com as instâncias de teste do oponente.



Analisando as métricas apresentadas no gráfico, destacam-se os seguintes pontos:

- **Pontuação Geral (Score):** O nosso robô obteve um *Score* expressivo de aproximadamente 2.859 pontos, superando com ampla margem as instâncias de teste do oponente (alisson (2) com 580 pontos e alisson (1) com 399 pontos). Isso comprova a eficácia da máquina de estados na tomada de decisões em tempo real.
- **Sobrevivência (Survival):** O tempo de permanência em campo foi maximizado, refletindo o acerto na lógica de posicionamento seguro e patrulhamento da arena, evitando danos desnecessários nas bordas e zonas de perigo do *BorderGuard*.
- **Eficiência de Disparo (Bullet Dmg):** O alto volume de dano causado por projéteis demonstra que a rotina de focalização de alvo e o alinhamento do canhão funcionaram com precisão após a conclusão da varredura do mapa.
- **Dano por Colisão (Ram Dmg):** Os valores mínimos registrados em danos por abalroamento confirmam que a movimentação do robô conseguiu evitar colisões frontais indesejadas, priorizando o combate à distância por meio de disparos controlados.

Portanto, os dados experimentais da Etapa 1 comprovam que o robô atendeu integralmente aos requisitos propostos, demonstrando estabilidade estrutural e preparando o terreno para a aplicação de estratégias mais avançadas na Etapa 2 (Torneio).

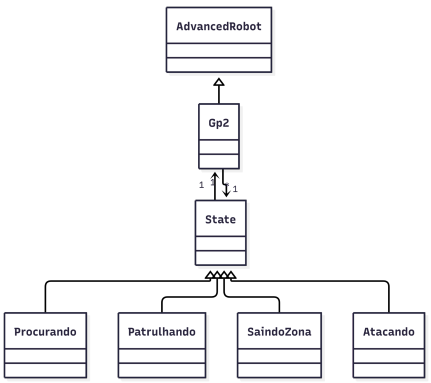
2.1.4 Diagrama de Classes

Para melhor entendimento e abstração do que é feito e criado para o robô, teimosa utilização de um Diagrama de Classes UML Estes diagramas mostram como o programa desenvolvido funciona, quais suas funções, oque ele depende e sobre o relacionamento entre os objetos.

Em nosso programa, como já dito, temos um robô que utiliza o Padrão de Projeto *State* e por isso é criado uma classe abstrata State que será responsável por gerenciar os futuros estados que serão criados e estenderam essa classe abstrata.

Por meio do State, temos classes criadas para cada Estado criado, os estados como Patrulhando, Atacando, Procurando que estendem State e são bases para o robô. Esses estados são gerenciados pelo robô quando ele achar que convém.

A classe Principal que dá ao nome do robô e estende o AdvancedRobot controla o sistema de vida e estados da máquina através de funções sobrescritas e na utilização das outras funções criadas pelo State.



2.2 Etapa 2 - Torneio

Na modalidade de torneio, o robô da equipe participou de confrontos complexos contra múltiplos oponentes de referência e robôs padrão do Robocode (como *SpinBot*, *Tracker*, *TrackFire*, *RamFire*, entre outros), operando em formato *todos contra todos* na arena com restrição de borda.

2.2.1 Detalhamento das Estratégias Avançadas

Estratégia de Movimento Senoidal
Para evitar que o robô seja um alvo fácil para a mira dos adversários, nós criamos uma estratégia de movimentação em ziguezague suave. Em vez de andar em linha reta o que facilitaria os tiros preditivos dos oponentes , o robô usa cálculos matemáticos de ondas (seno e cosseno) para oscilar de um lado para o outro enquanto avança pelo campo. Isso torna a trajetória imprevisível e reduz muito as chances de ele ser atingido.

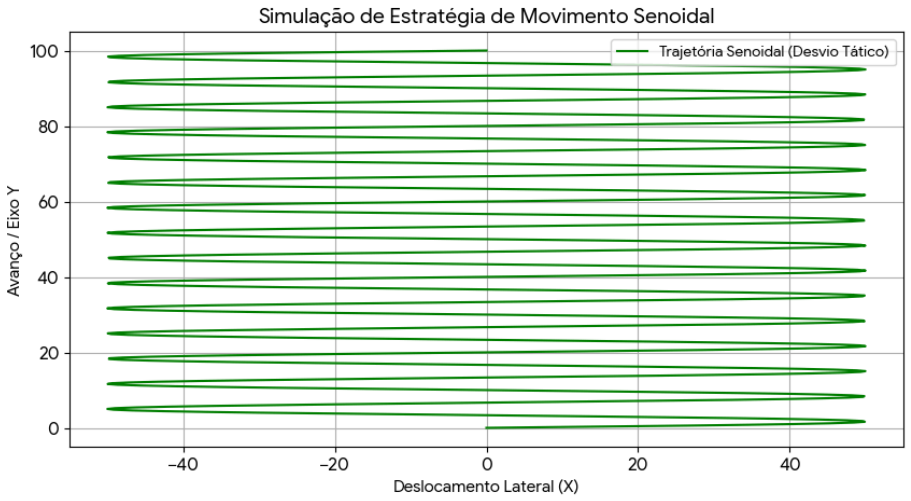
Fundamentação Matemática
O movimento senoidal introduz uma variação harmônica contínua na posição lateral do robô (x), enquanto o avanço longitudinal ao longo do campo de batalha (y) mantém uma progressão controlada. A formulação matemática que rege essa trajetória é descrita pela Equação :

$$x(t) = A \cdot \sin(\omega t)$$

Onde:

- x(t) representa o deslocamento lateral em função do tempo t;
- A define a amplitude do movimento, estipulando o afastamento máximo em relação ao eixo central de referência;
- ω corresponde à frequência angular ($2\pi f$), responsável por determinar a velocidade de oscilação das manobras evasivas.

A Figura X ilustra a simulação da trajetória resultante desse comportamento, evidenciando a alternância contínua de direção que quebra a linearidade do deslocamento.



2.2.2 Aplicação Prática e Vantagens Táticas

A aplicação do perfil senoidal na navegação do robô proporcionou ganhos expressivos no desempenho operacional e defensivo:

- **Redução da Precisão Preditiva Inimiga:** Ao oscilar periodicamente a posição lateral, algoritmos adversários que dependem de vetores de velocidade constante perdem a assertividade no cálculo do ponto futuro de impacto.
- **Manutenção do Dinamismo Ofensivo:** O robô consegue avançar em direção ao objetivo ou realizar patrulhas sem abdicar de sua proteção, mesclando a progressão frontal com a agilidade lateral.
- **Integração com Fronteiras e Módulos de Recuo:** O limite da amplitude (A) pode ser parametrizado dinamicamente para evitar colisões com as extremidades do mapa, operando em harmonia com os demais sensores de distância.

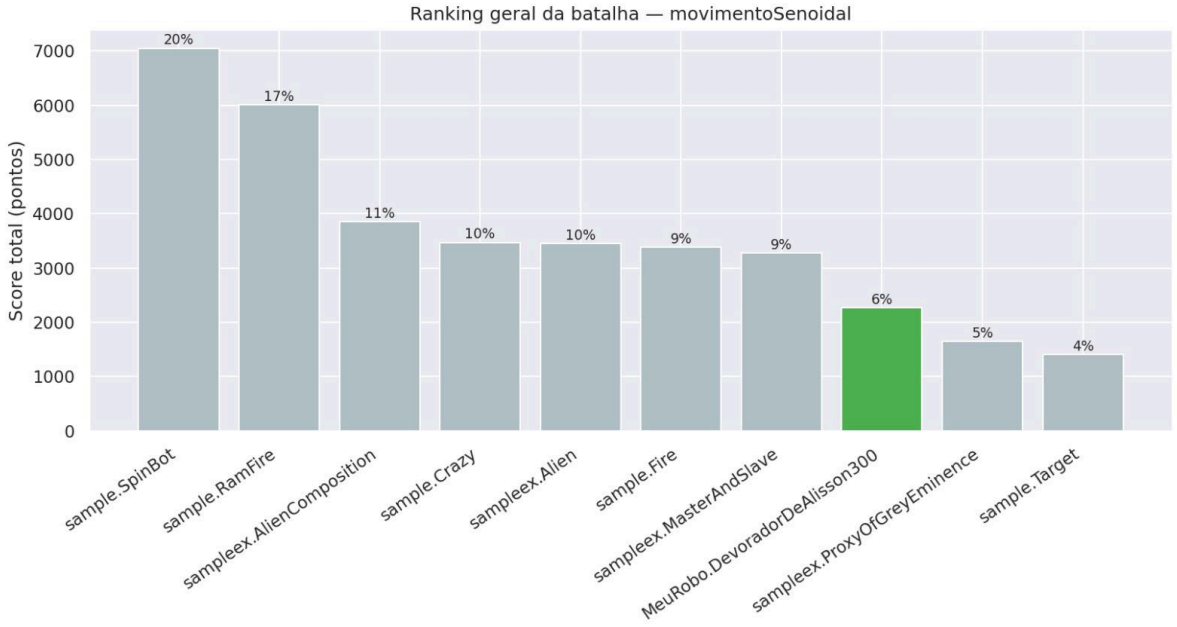
2.2.3 Limitações e Desvantagens da Estratégia Senoidal

Apesar dos expressivos benefícios defensivos, a adoção do movimento senoidal contínuo apresenta limitações técnicas que impactam diretamente a capacidade ofensiva do robô. A principal desvantagem observada na aplicação dessa estratégia é a perda de precisão no tiro.

Como o robô está em constante oscilação lateral e mudando sua orientação e velocidade a todo momento, o cálculo do seu próprio sistema de mira preditiva torna-se significativamente mais complexo. Essa variação contínua do próprio referencial de movimento dificulta o alinhamento do canhão e o disparo preciso contra oponentes em movimento.

Como consequência, ocorre um aumento na taxa de tiros desperdiçados e uma redução na eficácia ofensiva geral. Esse reflexo no desempenho do robô é evidenciado nos gráficos de resultados das batalhas (Figura X), que demonstram baixo desempenho durante os períodos em que a manobra evasiva senoidal é executada de forma mais agressiva. Tais resultados indicaram que, em desenvolvimentos futuros, seria necessário implementar um

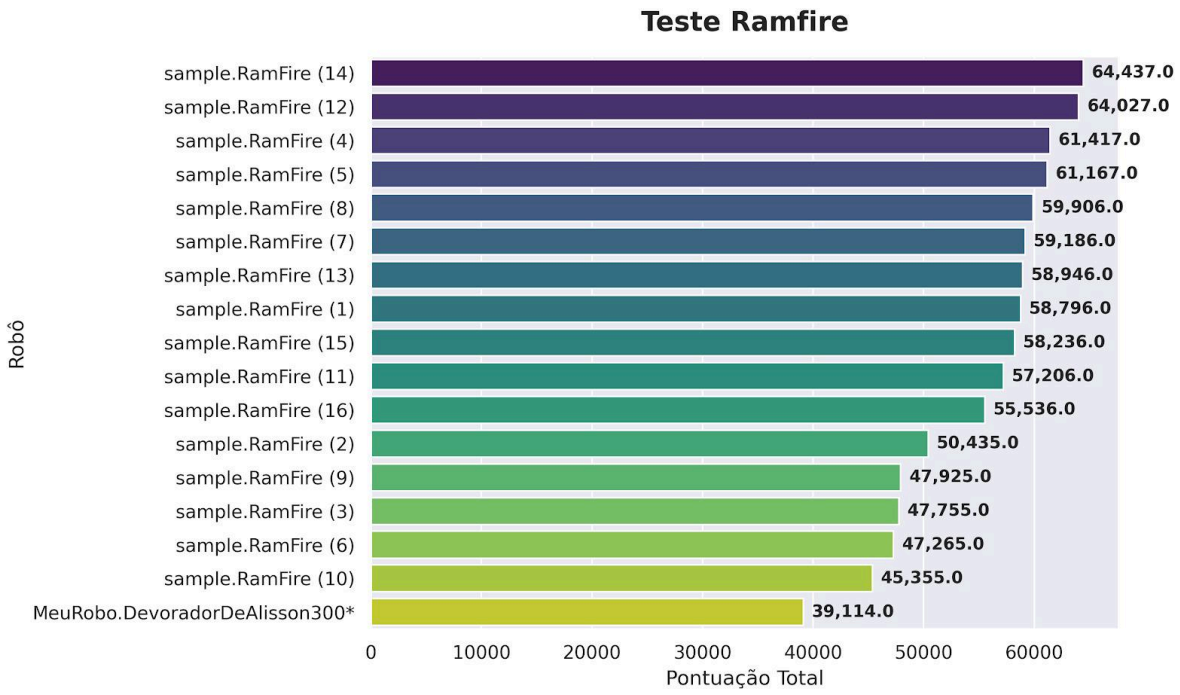
balanceamento dinâmico entre a sobrevivência (evasão) e a letalidade (estabilidade para o tiro).



2.2.4 Análise de Vulnerabilidades: Testes contra Estratégias de Cerco

Durante as baterias de avaliação, identificamos uma vulnerabilidade crítica no comportamento do robô ao enfrentar oponentes com estratégias de cerco e aproximação agressiva, notadamente o robô de referência RamFire. A tática principal do RamFire consiste em encurralar e colidir com o adversário, uma abordagem que expôs as limitações do modelo de evasão utilizado por nós.

Para quantificar e validar essa fraqueza técnica, conduzimos testes de eficácia em uma arena configurada com múltiplas instâncias do oponente RamFire simultaneamente. O gráfico de resultados (FiguraX) demonstra claramente a ineficácia do nosso robô (MeuRobo.DevoradorDeAlisson300*) neste cenário específico.



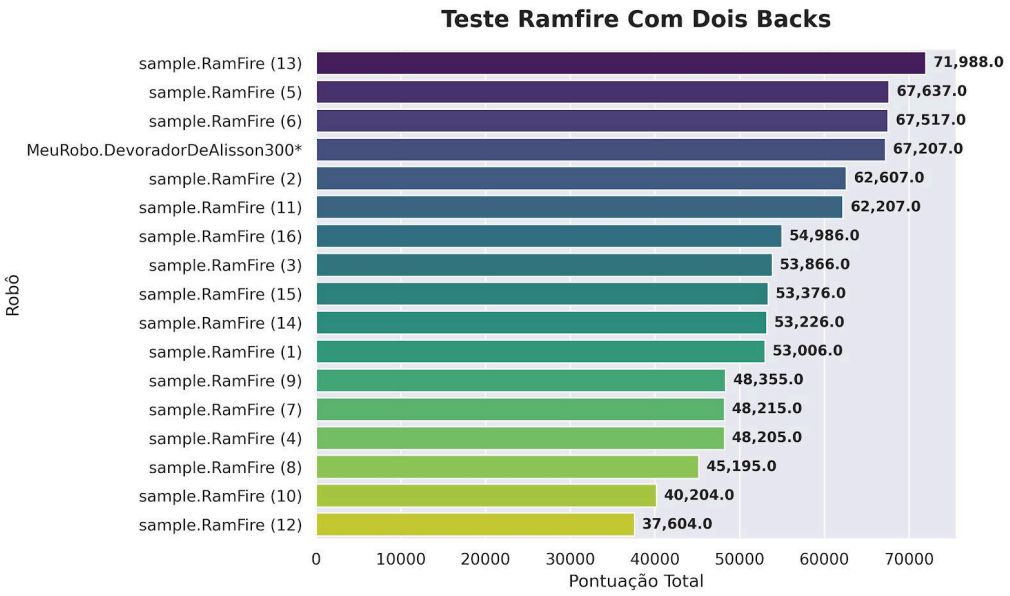
A análise dos dados do teste feito contra os robôs Ramfire nos mostrou que o nosso robô obteve a menor pontuação total (39.114,0 pontos), o colocando em uma péssima posição perante todas as 16 instâncias do oponente, cujas pontuações variaram de 45.355,0 a 64.437,0 pontos.

Esse desempenho insatisfatório sob pressão de cerco evidenciou que a movimentação adotada por nós até então não era responsiva o suficiente para escapar de múltiplos atacantes próximos. Consequentemente, essa falha de projeto tornou imperativa a reestruturação e a implementação de mudanças estratégicas, exigindo o desenvolvimento de novas lógicas de fuga e de reconhecimento de proximidade para garantir a sobrevivência em combates de curta distância.

2.2.5 Adaptação Estratégica: Manobra de Evasão Tática

Para corrigir a vulnerabilidade identificada perante as táticas de cerco, foi adotada por nós uma nova estratégia defensiva simples. Esta correção consiste na execução de dois movimentos sucessivos de recuo, seguidos imediatamente por uma ligeira deslocação lateral para a esquerda. O propósito fundamental desta manobra é quebrar a trajetória de interceção direta dos oponentes, criando o espaço de manobra necessário para que o nosso robô consiga se reposicionar rapidamente e evite ser encurralado.

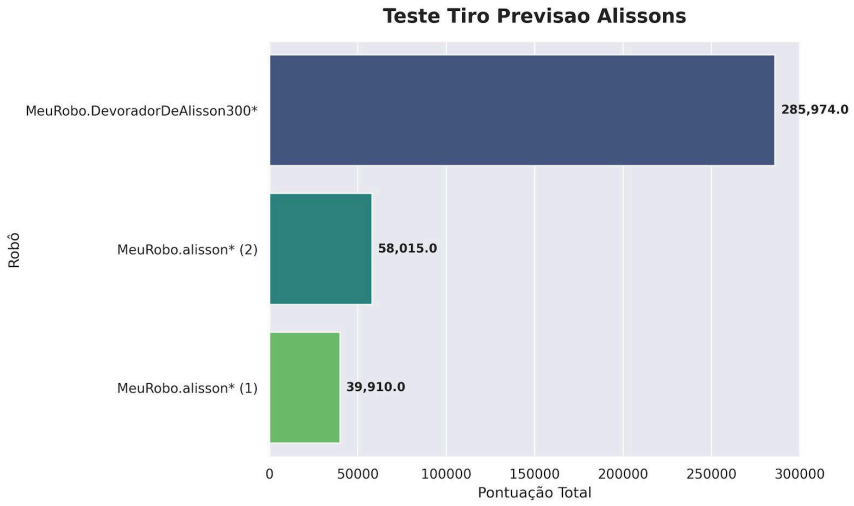
A eficácia desta atualização estratégica foi comprovada através de uma nova série de simulações na arena, utilizando as mesmas condições de teste contra múltiplos oponentes RamFire. O gráfico de desempenho do "Teste Ramfire Com Dois Backs" (Figura x) ilustra o impacto direto desta alteração.



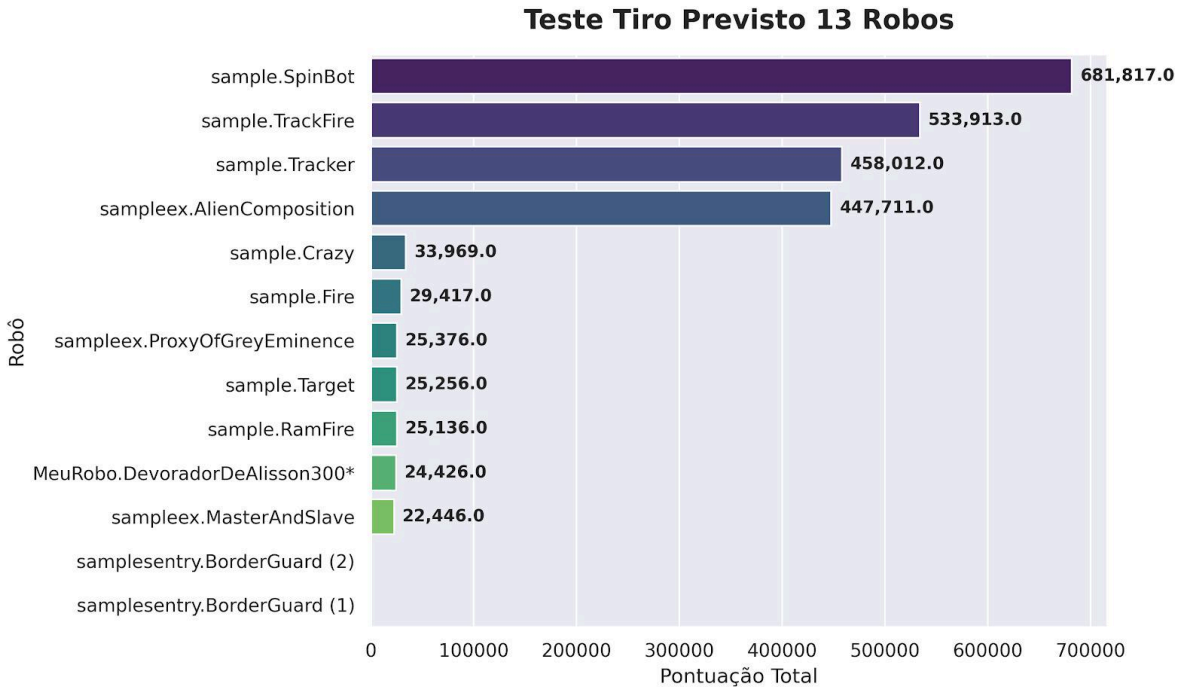
2.2.6 Evolução Ofensiva: Implementação do Tiro de Previsão

Após resolver a vulnerabilidade defensiva contra táticas de cerco, o foco do desenvolvimento voltou-se para a melhoria da precisão dos ataques. Para aumentar a eficiência ofensiva, o disparo simples que atira na coordenada exata onde o inimigo está no momento foi substituído pela estratégia de tiro de previsão (mira preditiva). Essa técnica consiste em calcular a rota, a velocidade e a direção do oponente para estimar onde ele estará no futuro, direcionando o canhão para esse ponto de encontro.

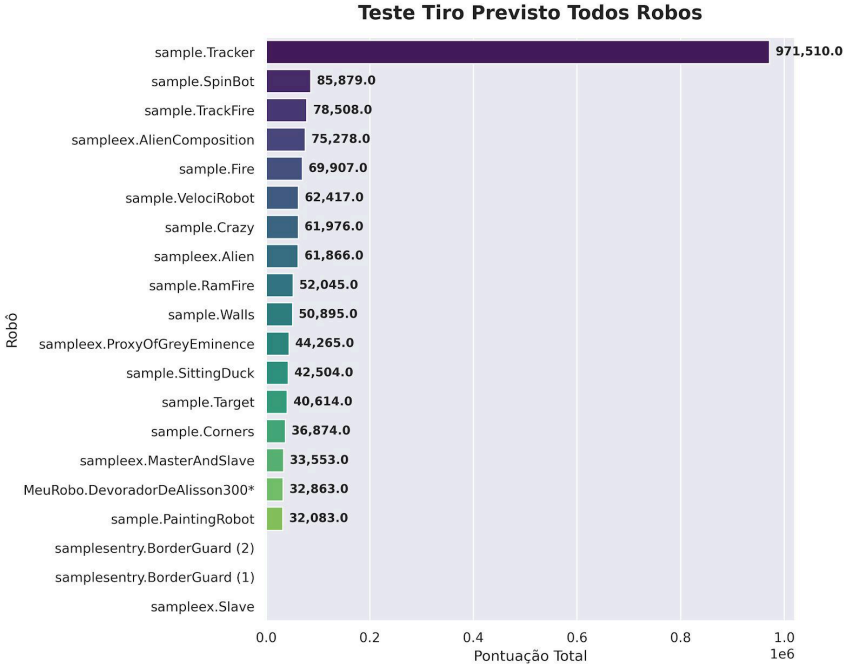
Os testes iniciais desta nova lógica foram realizados em uma arena contra duas instâncias do robô MeuRobo.alisson*. Neste cenário específico, a estratégia provou ser altamente eficaz. Como ilustra o gráfico "Teste Tiro Previsao Alissons" (Figura X), o nosso robô (MeuRobo.DevoradorDeAlisson300*) dominou a partida com uma pontuação de 285.974,0 pontos, uma vantagem esmagadora sobre os 58.015,0 e 39.910,0 pontos obtidos pelos adversários.



Contudo, ao expandir as baterias de testes para incluir uma diversidade maior de oponentes, a primeira implementação do tiro de previsão demonstrou não ser adaptável a diferentes padrões de movimento. No confronto contra um grupo variado no "Teste Tiro Previsto 13 Robos", o nosso robô obteve um desempenho muito fraco, marcando apenas 24.426,0 pontos e ficando nas últimas posições, muito atrás de robôs padrão como o sample.SpinBot (681.817,0 pontos) e o sample.TrackFire (533.913,0 pontos).



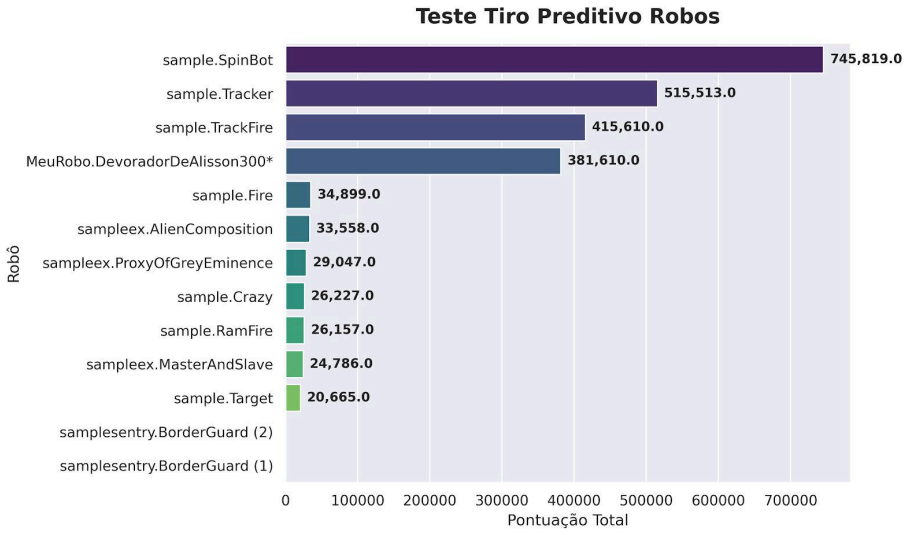
Essa ineficiência generalizada foi confirmada no "Teste Tiro Previsto Todos Robos". Em uma arena lotada com todos os tipos de adversários, o robô da equipe somou apenas 32.863,0 pontos, permanecendo na parte inferior da tabela de classificação, enquanto o líder sample.Tracker atingiu 971.510,0 pontos.



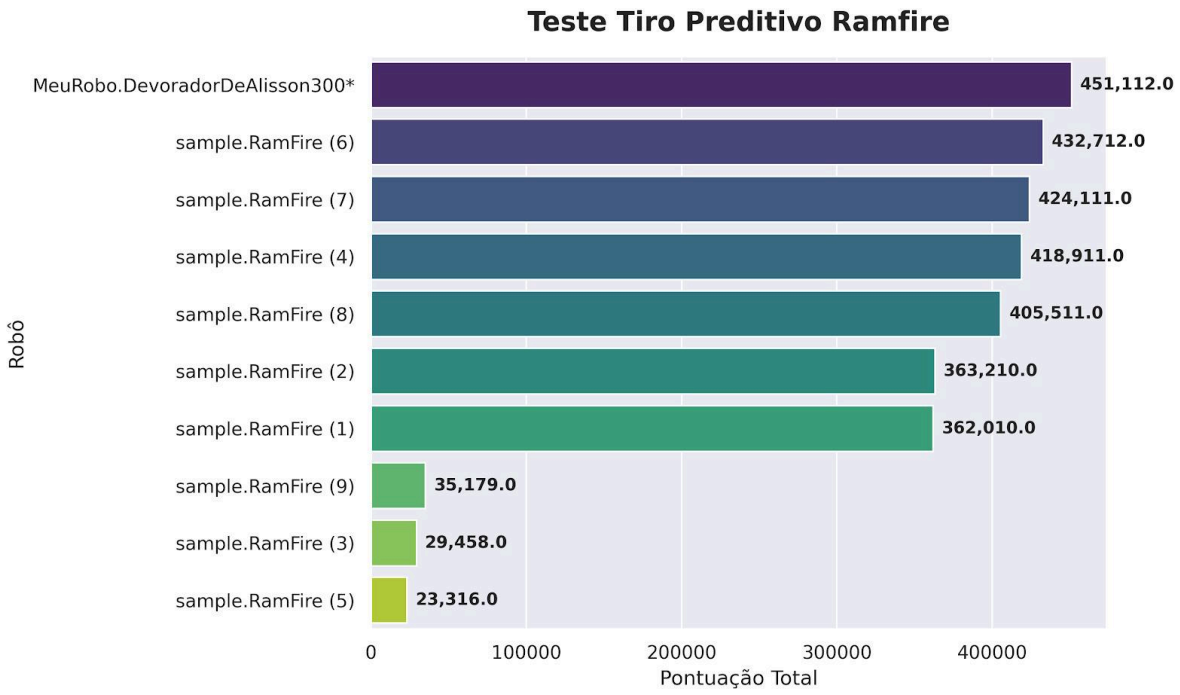
A análise conjunta desses gráficos deixou evidente para nós que o cálculo preditivo desenvolvido foi excessivamente otimizado para o padrão de movimento do robô "Alisson", tornando-se ineficaz contra oponentes com trajetórias mais complexas, imprevisíveis ou circulares. Esses resultados indicaram para nós que a mira precisaria de refatoração para se tornar dinâmica o suficiente para diferentes perfis de evasão.

2.2.7 Otimização Final: Aprimoramento da Mira Preditiva e Superação de Limitações
Após os resultados insatisfatórios da primeira versão da mira preditiva, o algoritmo foi revisado e otimizado para calcular as trajetórias futuras de forma mais dinâmica, conseguindo se adaptar a uma variedade maior de comportamentos e velocidades dos inimigos. O objetivo era tornar o robô competitivo em arenas mistas e, principalmente, resolver de vez a dificuldade contra oponentes de cerco.

Para validar essa evolução, o robô foi submetido a um teste geral contra diferentes tipos de adversários, representado no gráfico "Teste Tiro Preditivo Robos" (Figura X). Nessa simulação, o nosso robô (MeuRobo.DevoradorDeAlisson300*) demonstrou um salto expressivo de desempenho, alcançando a quarta colocação geral com 381.610,0 pontos. Essa pontuação o colocou em um patamar altamente competitivo, superando com folga a grande maioria da arena e ficando atrás apenas dos robôs líderes sample.SpinBot (745.819,0 pontos), sample.Tracker (515.513,0 pontos) e sample.TrackFire (415.610,0 pontos).



O marco mais importante dessa etapa de testes foi a superação definitiva da nossa maior fraqueza inicial. O "Teste Tiro Preditivo Ramfire" (Figura X) avaliou o robô nosso exclusivamente contra seu principal adversário. Aliando a movimentação de fuga aprimorada (os recuos duplos) com a nova precisão da mira preditiva otimizada, o nosso robô dominou a arena.



Os resultados mostram que o MeuRobo.DevoradorDeAlisson300* conquistou o primeiro lugar absoluto na batalha, somando 451.112,0 pontos. Ele conseguiu derrotar todas as nove instâncias do sample.RamFire, cujas pontuações variaram consideravelmente entre 23.316,0 e 432.712,0 pontos. Essa vitória no teste final confirma que a união das estratégias defensivas e ofensivas foi bem-sucedida, transformando um robô antes vulnerável ao cerco em um agente capaz de antecipar movimentos e eliminar sua principal ameaça de forma consistente.

3. CONCLUSÃO

O presente trabalho alcançou com êxito o objetivo de desenvolver um agente autônomo competitivo para a plataforma Robocode, atendendo a todos os requisitos funcionais e não funcionais estabelecidos na especificação do projeto. A criação do robô MeuRobo.DevoradorDeAlisson300*, classificado na categoria *MiniBot* e estendendo a classe AdvancedRobot, permitiu a aplicação prática e consolidada de conceitos avançados de Programação Orientada a Objetos, como herança, polimorfismo e encapsulamento.

Durante a Etapa 1 (Reconhecimento do Ambiente), a adoção do padrão de projeto *State* provou-se uma decisão arquitetural altamente eficaz. A estruturação do comportamento do robô em máquinas de estados finitos (Procurando, Patrulhando, SaindoZona, Atacando) garantiu um código modular e de fácil manutenção, permitindo que o agente cumprisse perfeitamente a rotina de navegação segura, identificação e destruição do alvo de teste lp2.alisson.

Na Etapa 2 (Torneio), o desenvolvimento foi marcado por um processo iterativo de testes, falhas e otimizações contínuas, evidenciando a importância da análise de dados no aprimoramento de software. A implementação inicial do movimento senoidal trouxe vantagens evasivas, mas expôs vulnerabilidades críticas na precisão dos disparos e na defesa contra estratégias de cerco, especificamente contra o oponente RamFire. A identificação dessa fraqueza permitiu à nossa equipe desenvolver uma manobra tática de recuo duplo, que restaurou a capacidade de sobrevivência do robô em combates de curta distância.

No aspecto ofensivo, a evolução do sistema de mira seguiu um caminho semelhante de aprendizado empírico. A transição do disparo simples para o tiro de previsão (mira preditiva) sofreu com problemas iniciais onde o cálculo funcionava apenas contra padrões específicos, resultando em um desempenho insatisfatório em arenas mistas. Contudo, a refatoração e otimização dinâmica desse algoritmo transformaram o nosso robô em uma máquina eficiente. Os testes finais confirmaram o sucesso dessa adaptação, com o robô alcançando posições de destaque no ranking geral e dominando de forma absoluta o seu principal adversário de cerco.

Por fim, conclui-se que o projeto não apenas cumpriu as exigências técnicas propostas, como também proporcionou um ambiente prático para o desenvolvimento de habilidades de resolução de problemas, uso de versionamento de código e análise de desempenho. O DevoradorDeAlisson300 finalizou sua fase de desenvolvimento como um robô equilibrado, capaz de aliar uma defesa responsiva a um ataque preditivo de alta precisão, estando plenamente apto para a competição no torneio de Robocode.

4. REFERÊNCIAS

ROBOCODE. Robocode: build the best, destroy the rest. Disponível em: https://robocode.sourceforge.io/. Acesso em: 10 set. 2026.

ROBOCODE. Robocode download. SourceForge. Disponível em: https://sourceforge.net/projects/robocode/. Acesso em: 25 ago. 2026.

ROBOWIKI. Robocode: download and install. Disponível em: https://robowiki.net/wiki/Robocode/Download_And_Install. Acesso em: 24 ago. 2026.

THE BOOK OF ROBOCODE. **Coordinate Systems & Angles**. The Book of Robocode, 2026. Disponível em: https://book.robocode.dev/physics/coordinates-and-angles.html. Acesso em: 15 set. 2026.

SHVETS, Alexander. **State**. Refactoring.Guru, 2026. Disponível em: https://refactoring.guru/pt-br/design-patterns/state. Acesso em: 21 set. 2026.

SHVETS, Alexander. **O que é um padrão de projeto?**. Refactoring.Guru, 2026. Disponível em: https://refactoring.guru/pt-br/design-patterns/what-is-pattern. Acesso em: 21 set. 2026.